

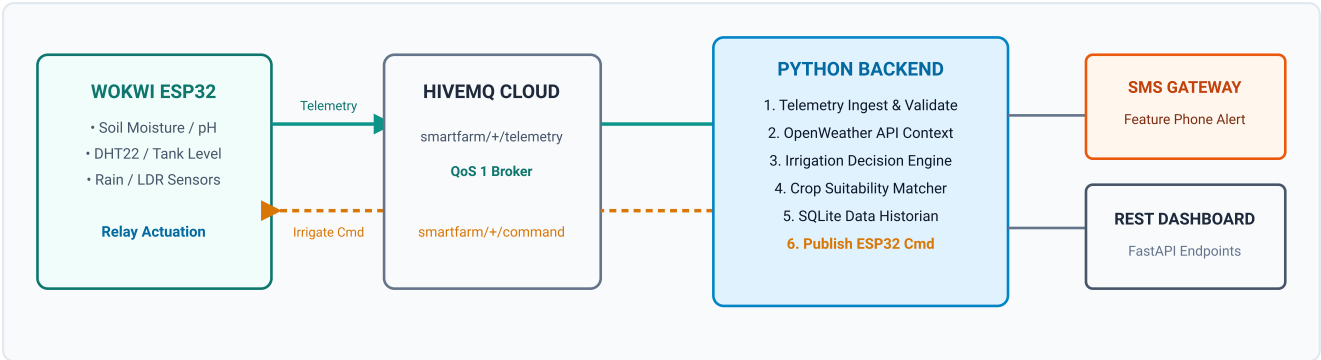
SmartFarm: Real-Time IoT Irrigation & Advisory

Closed-Loop MQTT Telemetry Ingestion, Predictive Decision Engine & Edge Actuation

PROTOCOL	BROKER TARGET	FRAMEWORK	DATA INGEST CYCLE
MQTT 3.1.1 (TLS 8883)	HiveMQ Cloud Cluster	Python 3.10+ / FastAPI	Real-Time Event Driven

1. Closed-Loop Real-Time Architecture

The system implements an autonomous closed loop. Wokwi ESP32 edge nodes capture local soil and atmospheric metrics, transmitting telemetry to HiveMQ Cloud. The Python backend subscribes to multi-tenant farm topics, ingests and validates the payload, combines field telemetry with weather forecasts, and publishes actionable control commands back to the ESP32 actuator relay.



Telemetry Flow (Upstream)

ESP32 samples sensors at 15s intervals, applies initial smoothing, packaging JSON payloads published to HiveMQ topic: `smartfarm/{FARM_ID}/telemetry`. Backend worker thread validates ranges and ingests readings.

Real-Time Actuation (Downstream)

When soil moisture drops below threshold AND rain forecast is low, backend emits an execution payload to `smartfarm/{FARM_ID}/command` instructing the ESP32 relay to engage the pump for a specified duration.

MQTT Topics, Payloads & Schemas

Bidirectional Communication Contracts for Wokwi Simulation and Physical Deployment

2. Topic Hierarchy & QoS Policies

Topic Template	Direction	QoS	Purpose & Trigger Policy
smartfarm/{farm_id}/telemetry	ESP32 → Back	1	Field sensor stream. Emitted every 15–30 seconds from ESP32 edge node.
smartfarm/{farm_id}/command	Back → ESP32	1	Actuator execution command (Pump ON/OFF, pulse duration).
smartfarm/{farm_id}/advisory	Back → Cloud	0	Real-time stream for web dashboard notifications and operational alerts.
smartfarm/{farm_id}/status	ESP32 ↔ Back	1	Heartbeat ping, pump feedback verification, and LWT (Last Will & Testament).

3. JSON Payload Schemas

1. Telemetry Ingest Schema (Subscribed)

```
{ "device_id": "FARM001",
  "timestamp": "2026-09-19T10:30:00Z",
  "soil_moisture": 24.5, // 0.0 to 100.0%
  "soil_ph": 6.4, // 0.0 to 14.0
  "temperature": 28.2, // Celsius
  "humidity": 68.0, // Relative %
  "light": 65.0, // Lux % level
  "rain": false, // Digital rain sensor
  "water_level": 78.0 // Tank reservoir % }
```

2. Actuation Command Schema (Published)

```
{ "command_id": "CMD_94821",
  "timestamp": "2026-09-19T10:30:05Z",
  "pump": true, // Actuator State: ON/OFF
  "duration_sec": 45, // Auto shutoff failsafe
  "reason": "Moisture 24.5% < threshold, Rain prob 10%",
  "force_override": false }
```

Actuator Fail-Safe Standard: Every pump: true command MUST carry a finite duration_sec. ESP32 firmware starts an internal timer to deactivate the relay automatically upon duration expiry. This prevents catastrophic flooding if connectivity drops while the pump is energized.

Decision Engine & Irrigation Logic

Validation, Soil Moisture Deficit, Weather Correlation & Actuation Pseudocode

4. Real-Time Irrigation Algorithm Pseudocode

```
ALGORITHM EvaluateAndActuateIrrigation(farm_id, telemetry_data): // Step 1: Range Validation IF NOT
IsValidTelemetry(telemetry_data) THEN LOG_WARNING("Corrupted packet received from: " + farm_id) RETURN //
Step 2: Extract Metrics moisture ← telemetry_data.soil_moisture tank_level ← telemetry_data.water_level
rain_sensor ← telemetry_data.rain // Step 3: Fetch Crop Thresholds and Weather Forecast crop ←
GetFarmerAssignedCrop(farm_id) // e.g., Maize (min_moisture: 35%) weather ←
FetchWeatherForecast(crop.farm_lat, crop.farm_lon) // Step 4: Decision Tree Analysis IF rain_sensor ==
TRUE THEN decision ← "STANDBY"; reason ← "Active rainfall detected at sensor." ELSE IF tank_level < 20.0
THEN decision ← "INHIBIT"; reason ← "Water reservoir level critical (< 20%).\" TriggerSmsAlert(farm_id,
"ALERT: Water tank dry. Pump disabled.") ELSE IF moisture < crop.min_moisture THEN IF
weather.rain_probability > 70 AND weather.rain_next_3h_mm > 4.0 THEN decision ← "DELAY"; reason ← "Soil
dry but rain forecast imminent (>70%).\" ELSE decision ← "IRRIGATE_START" duration ←
CalculatePumpDuration(moisture, crop.min_moisture) reason ← "Moisture below threshold (" + moisture +
"%), no rain expected.\" ELSE decision ← "STANDBY"; reason ← "Moisture optimal for " + crop.name // Step
5: Command Publishing to ESP32 IF decision == "IRRIGATE_START" THEN cmd_payload ← {"pump": true,
"duration_sec": duration, "reason": reason} MqttPublish("smartfarm/" + farm_id + "/command", cmd_payload)
AuditLogIrrigation(farm_id, "PUMP_ON", duration, reason) END IF
```

5. Anti-Flood SMS Advisory Throttling Logic

```
ALGORITHM DispatchSmsIfNecessary(farmer_id, new_advisory): last_sms ← QueryLastSmsSent(farmer_id)
now ← GetCurrentTimestamp() cooldown_seconds ← 14400 // 4-hour window for routine messages IF
new_advisory.severity == "CRITICAL" THEN SendSmsViaGateway(farmer_id.phone, new_advisory.text)
RecordSmsAudit(farmer_id, new_advisory, "SENT_EMERGENCY") ELSE IF (now - last_sms.timestamp) >
cooldown_seconds THEN IF new_advisory.digest != last_sms.digest THEN SendSmsViaGateway(farmer_id.phone,
new_advisory.text) RecordSmsAudit(farmer_id, new_advisory, "SENT_PERIODIC") END IF END IF
```

Project Structure & Data Model

Modular Enterprise Directory Tree, Database Tables & REST Interface

Folder Architecture

```
smartfarm-backend/ ├── app/ | ├── api/
# FastAPI routers | | ├── farms.py # Farm &
# farmer config | | └── telemetry.py # Historical
queries | ├── mqtt/ # HiveMQ Client | | ├──
client.py # Connection & loops | | ├──
topics.py # Topic constants | | └── handlers.py
# Ingestion callbacks | ├── engine/ # Domain
Logic | | ├── irrigation.py # Decision engine |
| ├── advisor.py # Crop advisor | | └──
weather.py # OpenWeather sync | ├── database/ #
Storage Layer | | ├── connection.py # SQLAlchemy
session | | └── models.py # ORM entities | └──
sms/ # Notification Layer | └── gateway.py #
Africa's Talking / Twilio ├── .env # Secrets &
Broker URL ├── main.py # Application Entry
Point └── requirements.txt # Locked
Dependencies
```

Relational Schema (SQLite / Postgres)

Core database tables managed via SQLAlchemy:

- **farmers:** id, name, phone, crop_id, lang
- **sensor_readings:** id, farm_id, timestamp, moisture, ph, temp, hum, rain, tank_level
- **irrigation_events:** id, farm_id, timestamp, duration_sec, trigger_reason, pump_state
- **sms_logs:** id, farm_id, message, sent_at, delivery_status

Indexing Strategy: Add composite indexes on (farm_id, timestamp DESC) in sensor_readings to ensure sub-5ms lookup for live dashboards.

6. REST Endpoints for Dashboard Consumption

Method	Endpoint	Description
GET	/api/farms/{farm_id}/latest	Returns most recent sensor telemetry and edge status packet.
GET	/api/irrigation/{farm_id}/history	Retrieves historical pump activation log with duration and reasons.
POST	/api/irrigation/{farm_id}/override	Manual operator override. Publishes immediate actuation command.

Team Startup Runbook & Verification

Step-by-Step Setup, HiveMQ Integration, and Closed-Loop Verification

7. Environment Setup & Dependencies

Execute the following commands to initialize the isolated Python environment:

```
# 1. Clone repository and create virtual environment git clone https://github.com/your-org/smartfarm-backend.git && cd smartfarm-backend python -m venv venv && source venv/bin/activate # On Windows: venv\Scripts\activate # 2. Install production dependencies pip install fastapi uvicorn paho-mqtt requests sqlalchemy pydantic python-dotenv
```

8. Environment Configuration (.env)

```
MQTT_BROKER_HOST=YOUR_HIVEMQ_CLUSTER.hivemq.cloud MQTT_BROKER_PORT=8883
MQTT_USERNAME=smartfarm_backend_user MQTT_PASSWORD=YourSecureHiveMQPassword123
DATABASE_URL=sqlite:///./smartfarm.db WEATHER_API_KEY=your_openweathermap_api_key
SMS_GATEWAY_API_KEY=your_africastalking_key
```

9. Verification Runbook (5-Step Validation)

Step	Phase	Validation Procedure & Success Criteria
1	Database Init	Run <code>python -m app.database.connection</code> to create SQLite tables. Verify <code>smartfarm.db</code> exists.
2	Backend Boot	Start FastAPI & MQTT subscriber: <code>uvicorn app.main:app --reload --port 8000</code> . Look for log: <code>Connected to HiveMQ Cloud (QoS 1)</code> .
3	Wokwi Ingestion	Run ESP32 in Wokwi. Check backend logs: <code>[INGEST] FARM001: Moisture=24.5% pH=6.4</code> written to DB.
4	Loop Closure	Lower potentiometer/soil in Wokwi to 22%. Verify backend publishes to <code>smartfarm/FARM001/command</code> . Confirm Wokwi LED/relay triggers ON.
5	Dashboard Test	Open <code>http://localhost:8000/docs</code> and execute <code>/api/farms/FARM001/latest</code> to confirm real-time UI data.